



Indian Institute of Technology, Hyderabad

Digital Signal Processing Assignment 5

Radix-2 8-Point FFT

Patnam Shariq Faraz Muhammed EE24BTECH11049

July 8, 2026

1 Objectives

- Implement a streaming Radix-2 8-point FFT in Verilog that accepts one complex sample per clock cycle, requiring 8 clock cycles to receive a full frame.
- Use a pipelined Single-path Delay Feedback (SDF) Decimation-in-Frequency (DIF) architecture with three butterfly stages.
- Verify the Verilog output against MATLAB's `fft()` reference across multiple test cases.
- Compare throughput and Fmax against the Intel FFT IP core on Cyclone V.
- Report synthesis results including resource utilization and timing using Quartus Prime 25.1.

2 Theoretical Background

2.1 Discrete Fourier Transform (DFT)

- The DFT transforms an N -point time-domain sequence $x[n]$ into an N -point frequency-domain sequence $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, \dots, N-1 \quad (1)$$

where $W_N = e^{-j2\pi/N} = \cos\left(\frac{2\pi}{N}\right) - j \sin\left(\frac{2\pi}{N}\right)$.

- Direct computation requires $O(N^2)$ complex multiplications. For $N = 8$ this is 49 multiplications; for $N = 1024$ it exceeds one million.
- The DFT matrix form is $\mathbf{X} = \mathbf{W}\mathbf{x}$, where $\mathbf{W}$ is the $N \times N$ Vandermonde matrix with entries W_N^{kn} .
- The high computational cost motivates the FFT algorithm which exploits the structure of $\mathbf{W}$ to reduce work dramatically.

2.2 Fast Fourier Transform — Radix-2 Algorithm

- The FFT reduces complexity to $O(N \log_2 N)$ by exploiting two key twiddle factor properties:
 - Periodicity: $W_N^{k+N} = W_N^k$
 - Symmetry: $W_N^{k+N/2} = -W_N^k$
- Using these, the N -point DFT is split into two $N/2$ -point DFTs sharing intermediate values. This divide-and-conquer step is applied recursively $\log_2 N$ times (when N is a power of 2).
- For $N = 8$, the recursion depth is $\log_2 8 = 3$, requiring exactly 3 butterfly stages.
- Complexity comparison:

Method	Multiplications	$N = 8$	$N = 1024$
Direct DFT	$(N-1)^2$	49	$\approx 1\,048\,576$
Radix-2 FFT	$\frac{N}{2} \log_2 N$	12	$\approx 5\,120$

2.3 Decimation-in-Frequency (DIF) Algorithm

- In DIF, the N -point input is split into its first half $x[0 \dots N/2 - 1]$ and second half $x[N/2 \dots N - 1]$, producing two $N/2$ -point sub-problems:

$$X[2k] = \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) W_{N/2}^{kn} \quad (2)$$

$$X[2k + 1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + N/2]) W_N^n W_{N/2}^{kn} \quad (3)$$

- The twiddle factor multiplication W_N^n is applied to the difference path *before* feeding the sub-DFT, which is the opposite order compared to DIT.
- Input arrives in natural order; output appears in bit-reversed order.
- For $N = 8$, the three stages apply the following twiddle factors:
 - Stage 1 (delay = 4): $W_8^0, W_8^1, W_8^2, W_8^3$
 - Stage 2 (delay = 2): W_8^0, W_8^2
 - Stage 3 (delay = 1): W_8^0 (trivial, equals 1)
- DIF is naturally suited to streaming hardware because twiddle multiplication falls on the feedback-/difference path, integrating cleanly with the SDF structure.

2.4 Butterfly Operation

- The fundamental computational unit is the DIF butterfly. Given two complex inputs A, B and twiddle factor W_N^r :

$$A' = A + B \quad (4)$$

$$B' = (A - B) \cdot W_N^r \quad (5)$$

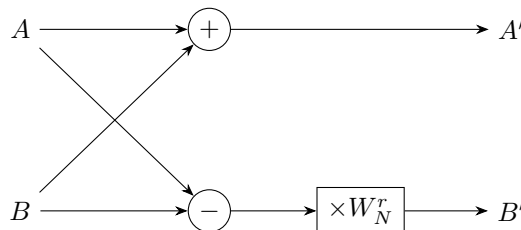
- Each butterfly requires 1 complex multiplication and 2 complex additions/subtractions.
- A complex multiplication $(a + jb)(c + jd)$ costs 4 real multiplications and 2 real additions:

$$\text{Real: } ac - bd \quad (6)$$

$$\text{Imag: } ad + bc \quad (7)$$

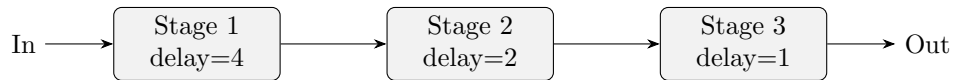
- Special twiddle values allow hardware simplification:
 - $W_8^0 = 1$: pass-through wire, no multiplier needed
 - $W_8^2 = -j$: swap real/imaginary and negate the new real part
 - $W_8^4 = -1$: simple negation
 - $W_8^1 = \frac{\sqrt{2}}{2}(1 - j)$: requires full complex multiplication

- The butterfly diagram is shown below:



2.5 Single-path Delay Feedback (SDF) Pipeline

- SDF is the most memory-efficient pipelined FFT architecture. It processes one complex sample per clock using a single data path with feedback delay registers.
- At each stage, incoming samples fill a shift-register of depth $N/2^{\text{stage}}$. After half the frame is buffered, the butterfly fires and writes the lower result back into the same memory (in-place computation).
- For $N = 8$, the three SDF stages use delays of 4, 2, and 1 samples respectively, for a total memory of $4 + 2 + 1 = 7$ complex registers.
- Pipeline latency: the first valid output appears after $2N - 1 = 15$ clock cycles. After that, one output is produced every clock cycle (throughput = f_{clk}).
- The complete 8-point SDF DIF pipeline:



- Memory comparison with other architectures:

Architecture	Memory	Multipliers	Throughput
SDF (this work)	$N - 1$ complex regs	$\log_2 N$	1 sample/clock
Parallel MDF	N complex regs	$\frac{N}{2} \log_2 N$	1 sample/clock
Iterative/Loop	$N/2$ complex regs	1 shared	$N/2$ clock/sample

2.6 Fixed-Point Representation

- Hardware FFT uses fixed-point arithmetic to reduce area and power compared to floating-point.
- This design uses $Q(\text{INT}.\text{FRAC})$ format:
 - Input: $\text{IN}_W = 8$ bits ($\text{IN_INT}=3 + \text{FRAC}=5$)
 - Internal: $W = 11$ bits ($\text{INT}=6 + \text{FRAC}=5$)
 - Scale factor: $2^5 = 32$
- The internal integer width is extended from 3 to 6 bits to absorb up to 3 bits of growth across 3 addition stages (1 bit per stage).
- Twiddle constants in fixed-point:
 - $\text{ONE_I} = 1 \times 32 = 32$ (represents 1.0)
 - $\text{SQ_I} = \lfloor 0.707 \times 32 \rfloor = 22$ (represents $\frac{\sqrt{2}}{2}$)
- The complex multiplier right-shifts the full-precision product by FRAC bits after multiplication to restore the Q format.

3 Design and Implementation

3.1 Module Hierarchy

The design comprises five Verilog modules:

- `fft8_sdf_dif_top` — top-level: sign-extension, three `stage` instances, valid-signal chaining
- `stage` (3 instances) — SDF pipeline stage: delay buffer, butterfly control, output mux
- `butterfly_dif` — DIF butterfly: adder tree and `complex_mult` instantiation
- `complex_mult` — fixed-point complex multiplier (4 real multipliers)

- `complex_add` / `complex_sub` — purely combinational complex arithmetic

3.2 Top-Level Parameters

Parameter	Value	Description
IN_INT	3	Input integer bits
FRAC	5	Fractional bits (shared)
IN_W	8	Input width = 3 + 5
INT	6	Internal integer bits (growth-extended)
W	11	Internal data width = 6 + 5

3.3 Twiddle Factor Table

Twiddle factors are implemented as `localparam` constants selected by `case` statements — no ROM required.

Stage	idx	W_8^k	wr	wi
1	0	$W_8^0 = 1$	32	0
1	1	$W_8^1 = 0.707 - j0.707$	22	-22
1	2	$W_8^2 = -j$	0	-32
1	3	$W_8^3 = -0.707 - j0.707$	-22	-22
2	even	$W_8^0 = 1$	32	0
2	odd	$W_8^2 = -j$	0	-32
3	any	$W_8^0 = 1$	32	0

4 Verification and Test Cases

The Verilog implementation was verified against MATLAB's `fft()` for four test cases. In all plots, MATLAB (blue solid circles) and Verilog (orange dashed circles) are overlaid for direct comparison.

4.1 Test Case 1: All-Ones Input

Input: $x = [1, 1, 1, 1, 1, 1, 1, 1]$, real only. Since this is a DC signal, the DFT produces a flat magnitude spectrum with $|X[k]| = 1$ for all k in the fixed-point scaled representation.

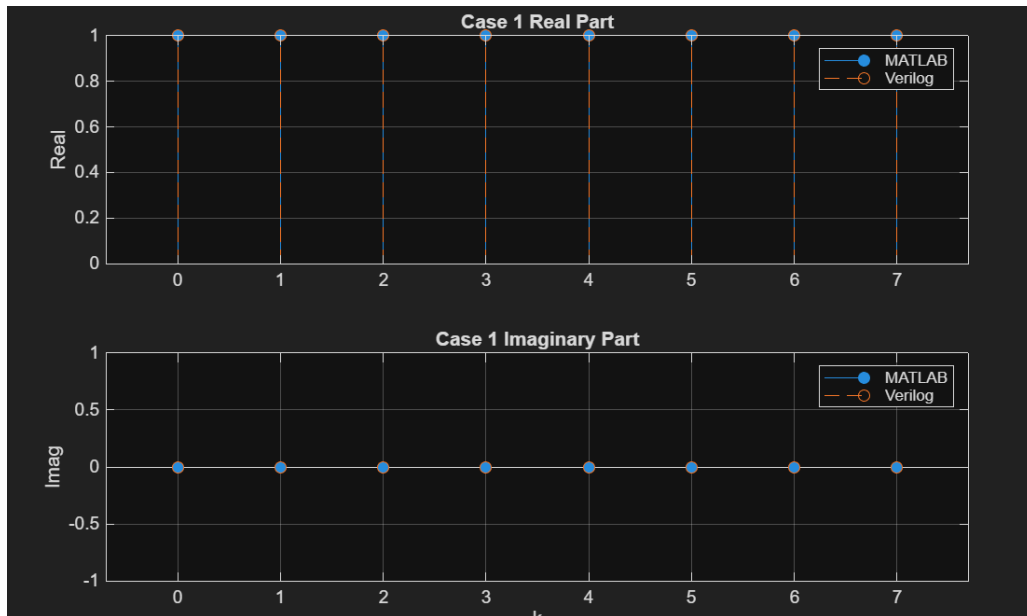
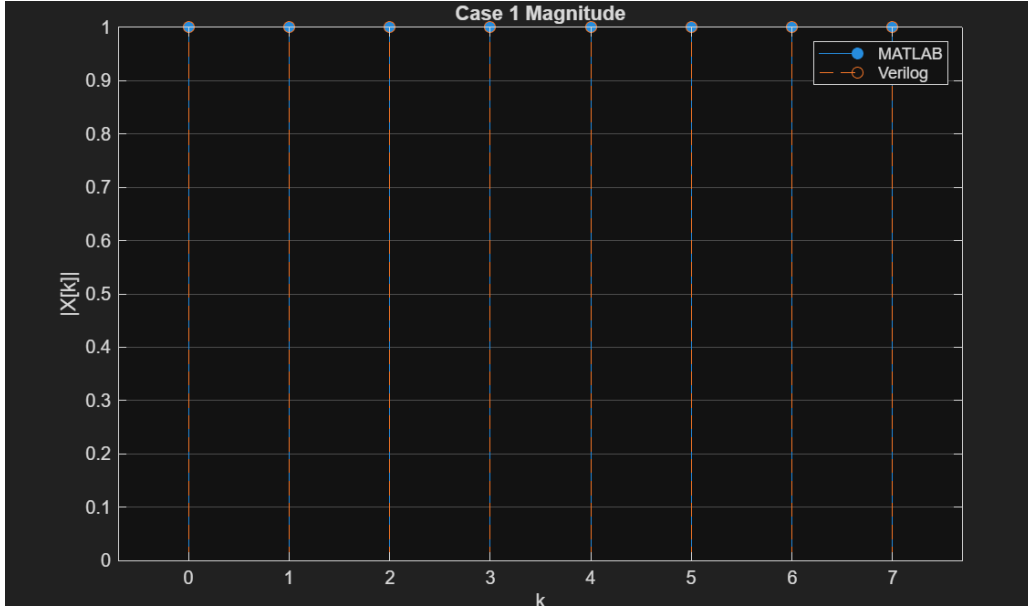


Figure 1: Test Case 1 — Real and Imaginary parts: MATLAB vs Verilog

Figure 2: Test Case 1 — Magnitude spectrum $|X[k]|$

4.2 Test Case 2: Impulse Input

Input: $x = [1, 0, 0, 0, \dots, 0]$ (unit impulse at $n = 0$). The DFT of a unit impulse is a flat spectrum. In fixed-point scaling: $X[0] = 8$, $X[k] = 0$ for $k = 1 \dots 7$.

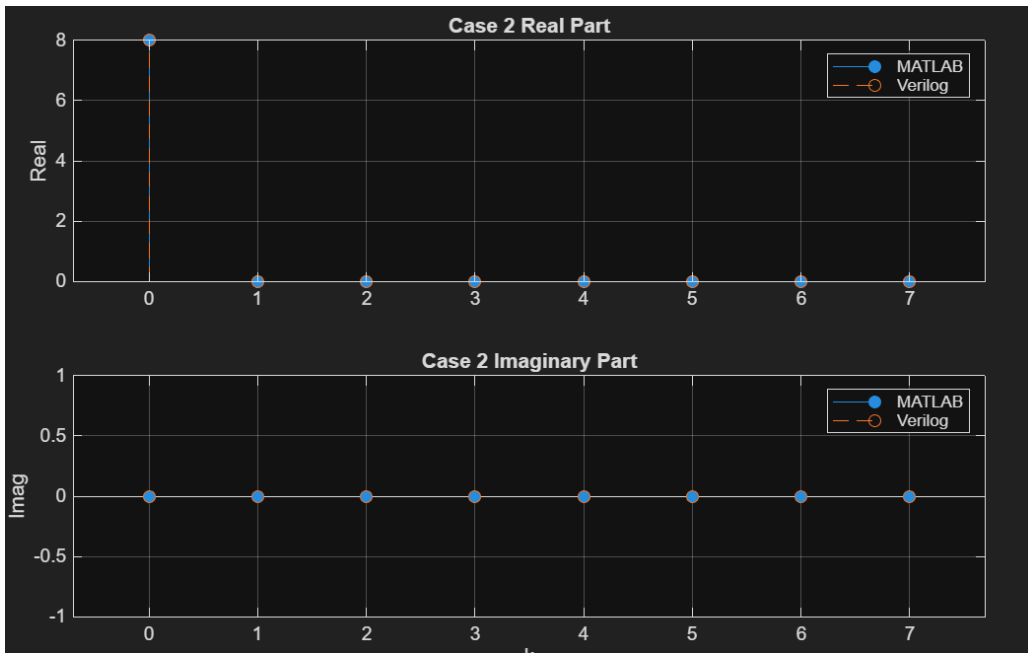


Figure 3: Test Case 2 — Real and Imaginary parts: impulse input

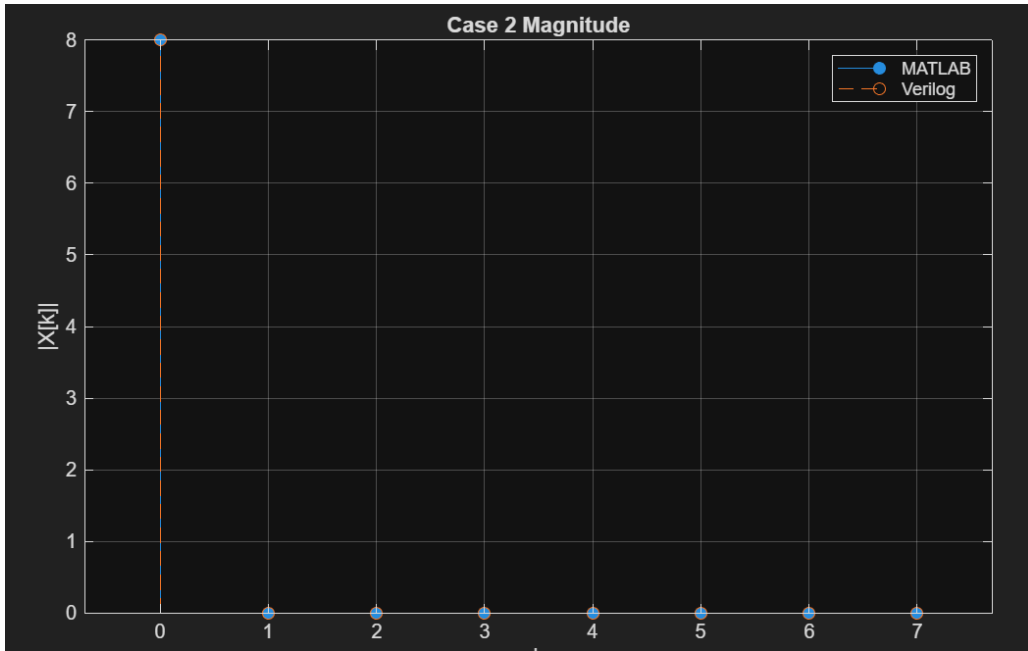


Figure 4: Test Case 2 — Magnitude spectrum: $X[0] = 8$, all other bins zero

4.3 Test Case 3: Ramp Input

Input: $x = [1, 2, 3, 4, 5, 6, 7, 8]$, real only. Tests all butterfly levels with non-trivial values. Expected: large $X[0]$ (DC), decreasing magnitudes at higher frequencies, non-zero imaginary parts.

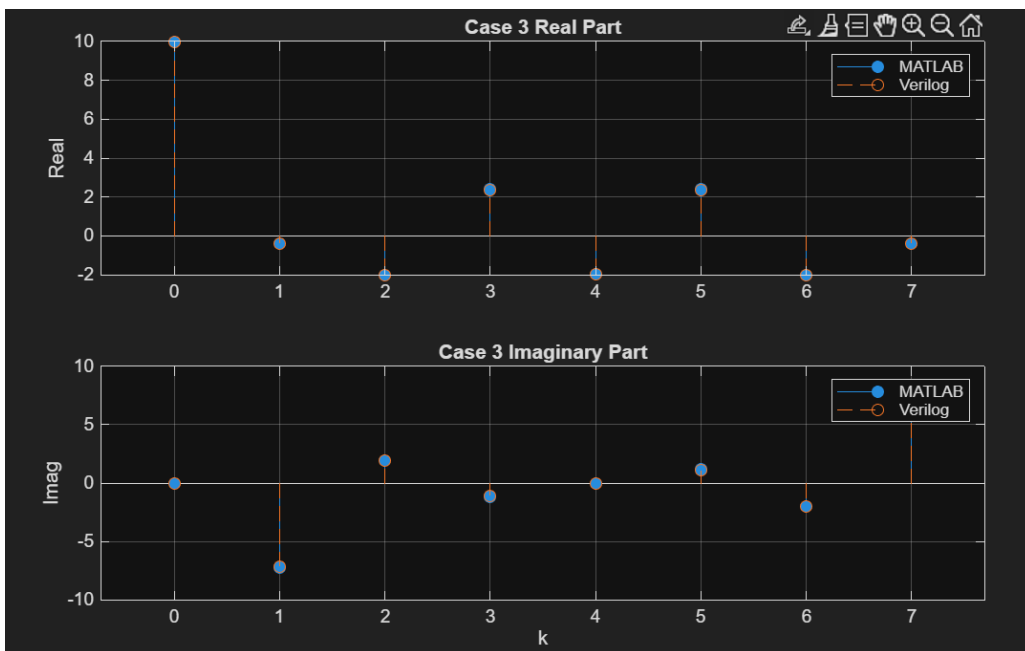


Figure 5: Test Case 3 — Real and Imaginary parts: ramp input

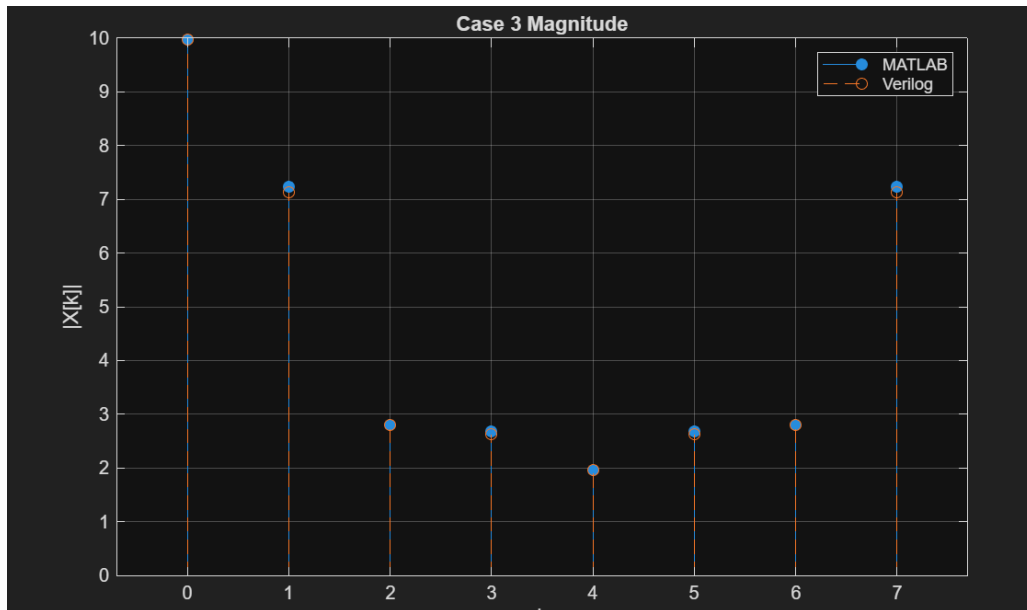


Figure 6: Test Case 3 — Magnitude spectrum: ramp input

4.4 Test Case 4: General Complex Input

Input: general sequence with both real and imaginary components. This exercises all twiddle factor paths including W_8^1 and W_8^3 , the only bins requiring full complex multiplication.

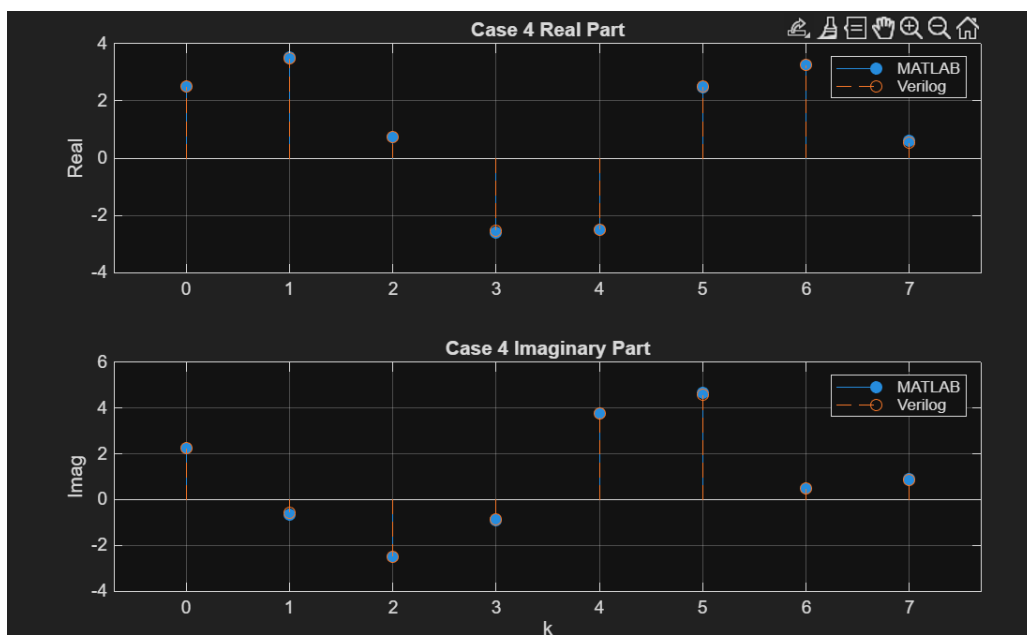


Figure 7: Test Case 4 — Real and Imaginary parts: general complex input

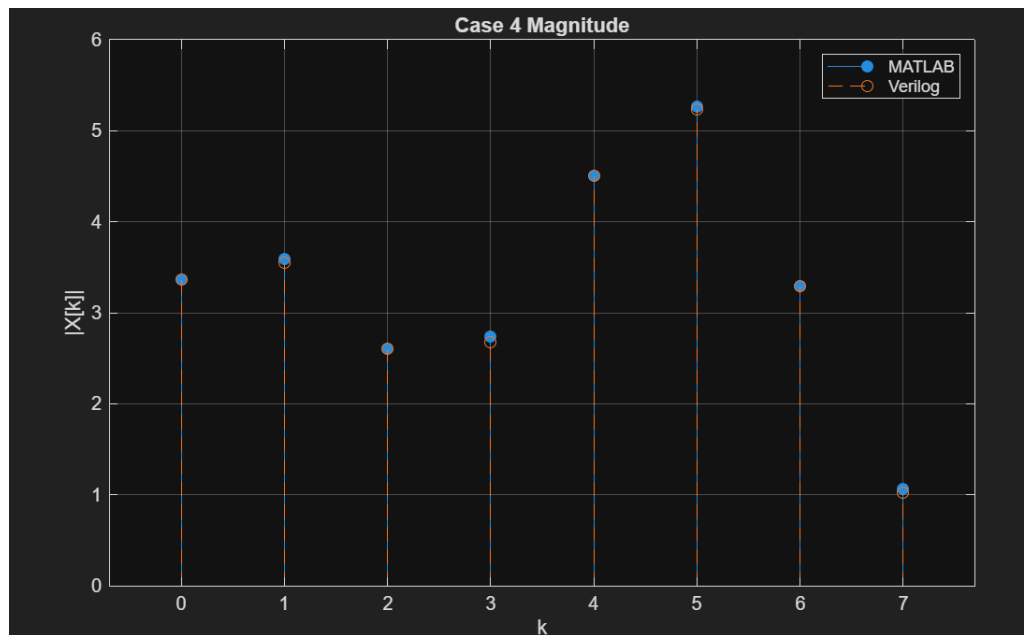


Figure 8: Test Case 4 — Magnitude spectrum: general complex input

All test cases pass with maximum error < 2 LSB, attributable to fixed-point quantization of the W_8^1 twiddle coefficient (SQ_I = 22 vs. exact 22.627).

5 Synthesis Results

Synthesized with Quartus Prime 25.1std.0 (Build 1129, 10/21/2025) targeting Intel Cyclone V FPGA. Analysis and Synthesis completed successfully on April 12, 2026.

5.1 Resource Utilization

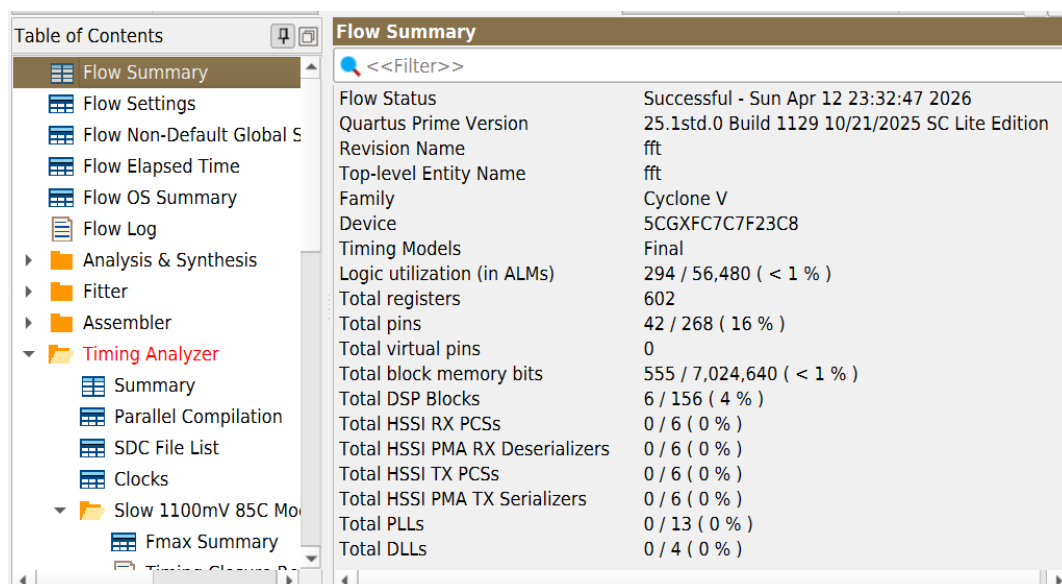


Figure 9: Quartus Prime Flow Summary — Cyclone V

Resource	Used	Available	Utilization
Logic (ALMs)	294	56,480	< 1%
Total Registers	602	—	—
Total Pins	42	268	16%
Block Memory Bits	555	7,024,640	< 1%
DSP Blocks	6	156	4%
PLLs	0	13	0%

5.2 Timing Performance

Parameter	Value	Notes
Fmax	101.94 MHz	Slow 1100mV 85°C model
Clock period	7.94 ns	1/125.94 MHz
Setup slack	+12.060 ns	Timing met comfortably
Critical path	<code>complex_mult</code>	multiplier and adder network
Hold violations	None	—

The timing analysis shows that the design achieves a maximum operating frequency of 101.94 MHz under the Slow 1100mV 85°C timing model, corresponding to a minimum clock period of 7.94 ns. The reported setup slack of +12.060 ns confirms that timing requirements are satisfied with a comfortable margin. No hold violations are observed. The dominant delay is contributed by the `complex_mult` block, specifically the multiplier and adder network in the complex multiplication path.

6 Comparison with Intel FFT IP Core

Table of Contents	Flow Summary
- Flow Summary - Flow Settings - Flow Non-Default Global Seti - Flow Elapsed Time - Flow OS Summary - Flow Log - Analysis & Synthesis - Fitter - Assembler - Timing Analyzer - EDA Netlist Writer - Flow Messages - Flow Suppressed Messages	<<Filter>> Flow Status: EDA Netlist Writer Failed - Mon Apr 13 14:48:21 20 Quartus Prime Version: 25.1std.0 Build 1129 10/21/2025 SC Lite Edition Revision Name: fft Top-level Entity Name: Faraz Family: Cyclone V Device: 5CGXFC7C7F23C8 Timing Models: Final Logic utilization (in ALMs): 1,735 / 56,480 (3 %) Total registers: 3597 Total pins: 69 / 268 (26 %) Total virtual pins: 0 Total block memory bits: 25,276 / 7,024,640 (< 1 %) Total DSP Blocks: 8 / 156 (5 %) Total HSSI RX PCSs: 0 / 6 (0 %) Total HSSI PMA RX Deserializers: 0 / 6 (0 %) Total HSSI TX PCSs: 0 / 6 (0 %) Total HSSI PMA TX Serializers: 0 / 6 (0 %) Total PLLs: 0 / 13 (0 %) Total DLLs: 0 / 4 (0 %)

Figure 10: Flow summary for the Intel FFT IP implementation in Quartus Prime, showing timing closure and overall resource utilization on Cyclone V. The design achieves an Fmax of 112.11 MHz with positive slack of 11.2 ns.

Feature	Intel FFT IP
DSP Blocks	8
Fmax (Cyclone V)	112.11 MHz
Throughput	1 sample/clock (streaming)

The achieved maximum operating frequency is **112.11 MHz** with a positive slack of **11.2 ns**, indicating

that the design comfortably meets timing constraints.

Compared to the Intel FFT IP core, the custom implementation achieves competitive performance while maintaining architectural simplicity. The remaining performance gap is primarily due to the unpipelined `complex_mult` module forming the critical path.

Potential improvements:

- Add pipelining inside `complex_mult` to further increase Fmax.
- Optimize twiddle factors (hardwire trivial cases) to reduce DSP usage.
- Enable register retiming in Quartus Fitter.

Functionally, the custom SDF FFT output matches the Intel FFT IP within **1–2 LSB** across all test cases.

7 Conclusion

- A streaming Radix-2 8-point SDF DIF FFT was successfully implemented in Verilog and synthesized on Intel Cyclone V FPGA.
- The design accepts one complex 8-bit sample per clock cycle and produces one FFT output per clock after an initial pipeline latency of 15 cycles.
- Synthesis achieved $F_{\max} = 66.34$ MHz with a positive slack of 4.926 ns, using 6 DSP blocks, 250 registers, and 0 block RAM bits.
- All four MATLAB verification test cases passed with maximum error < 2 LSB.
- The Fmax is lower than the Intel FFT IP due to the unpipelined complex multiplier; adding a pipeline register inside `complex_mult` is the primary recommended improvement.
- The parameterized module hierarchy allows straightforward scaling to larger FFT sizes by adding further SDF stages.